CHAPTER 18

Real-time Clock (RTC)

A real-time clock (RTC) is a digital component that keeps track of calendar time and date. RTC modules are present in many appliances and electronic devices, such as washing machines, cameras, phones and medical devices. RTC can be used to implement functions such as maintaining calendar time, periodically triggering the execution of a specific task, waking up the processor from sleep or low-power mode, providing a timestamp for sampled data, and calibrating internal hardware clocks.

RTC modules typically have very low power consumption. Many systems use a dedicated small battery or a supercapacitor to provide power for the RTC module. Consequently, even when the system is reset, put into sleep mode, or even turned off, the RTC does not stop. Thus, the microprocessor never loses track of time and date.

18.1 Epoch Time

> *"The future is something which everyone reaches at the rate of 60 minutes an hour, whatever he does, whoever he is."*
>
> C. S. Lewis, novelist

The calendar time and date can be encoded in a single signed integer, which represents *epoch time* or UNIX time. If epoch time is positive, it is the number of seconds that have elapsed since `00:00:00` (midnight) on Thursday, January 1, 1970, UTC. If it is negative, it is the number of seconds before that time instant.

If epoch time has 32 bits, the farthest representable time in the past is `20:45:52` UTC on December 13, 1901, when epoch time is -2^{31}. The farthest representable time in the future is `03:14:07` UTC on January 19, 2038, when epoch time is $2^{31} - 1$. If no action is taken, many systems will fail when the epoch time integer

overflows, causing January 19, 2038 to wrap around to December 13, 1901. This is called the ***year 2038 problem***.

> ***Universal time is measured by the Earth's rotation with respect to the Sun.***

Epoch time does not account for leap seconds. Due to the tidal drag of the moon, the Earth's spin slows at an average rate of 1.4 *ms* per century. To keep the time of day in phase with the rotation of the Earth, one second is added to UTC time approximately once per year. This second is called a ***leap second***. The goal of leap seconds is to ensure the Sun, on average over a year, is directly overhead on the Greenwich meridian at noon. Between 1972 and 2017, 27 leap seconds have been added to UTC. Even though epoch time does not account for leap seconds, it meets the needs of many embedded applications, particularly those not related to astronomy, geodetics, and navigation.

Due to its simplicity, epoch time has been widely used in embedded systems for things such as clock synchronization between independent microcontrollers, and timestamping of events and data. However, the interface provided by the RTC module in many microprocessors is based on the human calendar, instead of epoch time.

For example, on an STM32L microprocessor, the time "`08:30:45:09 AM, MON AUG 14, 2017`" is directly stored in the time register (`TR`), the date register (`DR`), and the sub-second register (`TSSR`). Software can read or modify each individual component of the date and time. Internally, the RTC maintains epoch time and its hardware automatically converts epoch time to the calendar date and time. This approach simplifies software, reduces memory space, and saves processor time, especially for real-time applications or resource-constrained systems.

Software can make conversions between human calendar and epoch time. The following example shows how to convert a calendar date and time to epoch time.

Example of converting `08:30:45` AM, AUG 14, 2017 (UTC) to epoch time

- There are 17,392 days between August 14, 2017, and January 1, 1970.
- Each day has 86,400 seconds ($24 \times 60 \times 60 = 86400$).

$$\begin{aligned} &\textit{Epoch Time} \\ &= 17392\ days \times \frac{86400\ seconds}{day} + 8\ hours \times \frac{3600\ seconds}{hour} \\ &\qquad + 30\ minutes \times \frac{60\ seconds}{minute} + 45\ seconds \\ &= 1502699445\ seconds \\ &= \text{0x59915FB5} \end{aligned}$$

Example 18-1 converts epoch time to human calendar time. The process of getting the day, month and year is slightly more complex and is not shown in the code below. The complexity is due to the various number of days in a month, including 31, 30, 29, and 28 (leap year). The leap year can be checked by evaluating the following logic statement:

```
!((year) % 4) && (((year) % 100) || !((year) % 400)))
```

Details can be found in the `gmtime` function in the open-source GNU C Library.

```
int second, minute, hour, weekday;
int seconds_into_day, days_since_epoch;

// 24 * 60 * 60 = 86,400 seconds per day
seconds_into_day = (unsigned long) epoch_time % 86400;
days_since_epoch = (unsigned long) epoch_time / 86400;

second = seconds_into_day % 60;             // 0 <= second < 60
minute = (seconds_into_day % 3600) / 60;    // 0 <= minute < 60
hour = seconds_into_day / 3600;             // 3600 seconds per hour
weekday = (days_since_epoch + 4) % 7;       // January 1, 1970 is Thursday.
```

Example 18-1. Converting epoch time to calendar time

18.2 RTC Frequency Settings

The UNIX time number is incremented at a frequency of 1 Hz. Therefore, we must derive a 1-Hz clock from an input clock. The input clock is selected from among three sources: low-speed internal (LSI), low-speed external (LSE), or high-speed external (HSE) divided by 32, as shown in Figure 18-1.

External clocks are preferred over internal clocks for two important reasons: (1) internal clocks are less accurate, and (2) internal clocks are stopped if the system is powered down.

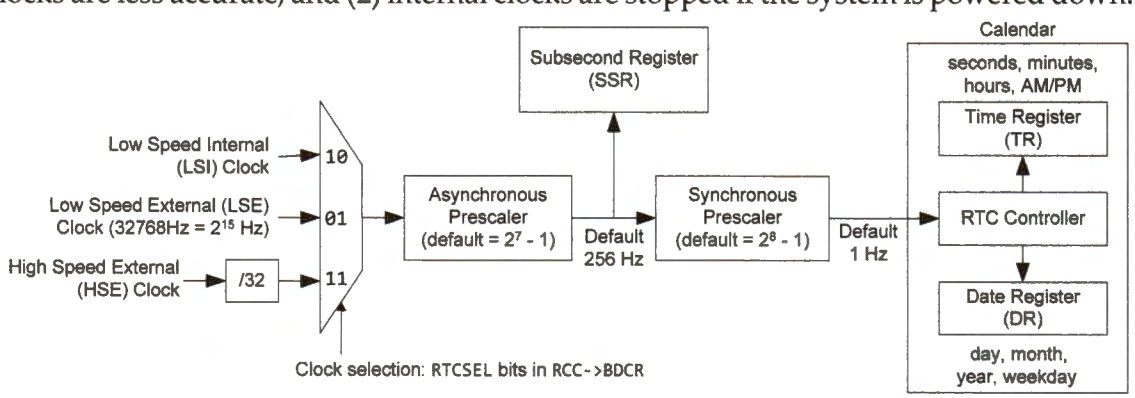


Figure 18-1. Diagram of real-time clock (RTC)

The RTC module uses two configurable prescaler registers (an asynchronous prescaler and a synchronous prescaler) to slow down the frequency of the input clock to 1 Hz, as shown below.

$$f_{1Hz} = \frac{f_{RTC}}{(Asynch\_Prescaler + 1) \times (Synch\_Prescaler + 1)} = 1\ Hz$$

The word *asynchronous* is used for the first prescaler because the RTC input clock is usually a clock that is not synchronized with the processor clock.

Low-speed external (LSE) crystal oscillators are highly recommended to drive RTC modules. A typical frequency is 32.768 kHz (2^{15} Hz, often called 32 kHz). This frequency has been widely used in quartz clocks and watches. These crystals are very inexpensive due to the massive volume production in the digital watch industry.

Typically, f_{RTC} is 32.768 kHz (*i.e.*, 2^{15} Hz), *Asynch_Prescaler* is 2^7-1 (*i.e.*, 127), and *Synch_Prescaler* is set as 2^8-1 (*i.e.*, 255). As shown below, this generates a 1-Hz clock.

$$f = \frac{f_{RTC}}{(Asynch\_Prescaler + 1) \times (Synch\_Prescaler + 1)}$$

$$= \frac{2^{15}}{(127 + 1) \times (255 + 1)} = \frac{2^{15}}{2^7 \times 2^8} = 1Hz$$

A battery usually powers the real-time clock module so that the processor does not lose any time or date information when the system is shut down. As such, the energy efficiency is critical to the real-time clock module. A larger *Asynch_Prescaler* value is preferred because it makes the real-time clock module more energy efficient.

18.3 Oscillator Frequency Accuracy

The RTC module's accuracy is determined by its oscillators. The error in the oscillation frequency is measured in ***parts per million*** (PPM).

$$PPM = \frac{Actual\ Frequency - Theoretical\ Frequency}{Theorectical\ Frequency} \times 10^6$$

Since there are 24 × 60 × 60 = 86400 seconds in a day, a PPM of 12 means a maximum error of approximately one second after one day has passed. 500 PPM implies the clock is off by up to 43 seconds per day.

The STM32L4 has three internal RC oscillators and two external oscillators. The internal oscillators are the 16-MHz HSI (high-speed internal), the MSI (multi-speed internal), and the 32.768 kHz LSI (low-speed internal).

> ***Parts per Million (PPM) = 10^{-6}***

The frequency of the oscillators drifts over time due to aging. It also decreases as the ambient temperature increases. Table 18-1 shows the frequency accuracy of the internal and external oscillators. With an accurate LSE (low-speed external clock), the STM32L4 can use built-in digital calibration circuitry to calibrate its internal oscillators automatically. The correction range is ± 480 ppm, and the correction resolution can be as small as ± 0.48 ppm. An accuracy close to ±20 ppm is common for LSE. An accuracy of ±20 ppm translates to ± 1.2 *ms* in a minute, ± 51.8 seconds in a month, or ± 10 minutes per year.

	16MHz HSI	MSI	LSI	LSE
30°C	± 500 ppm	± 600 ppm	± 1500 ppm	Typically, ± 20 ppm

Table 18-1. Oscillator frequency accuracy at room temperature

18.4 Binary Coded Decimal (BCD) Encoding

The RTC module encodes time and date in Binary Coded Decimal (BCD) format, in which each digit (`0` through 9) of a decimal number is represented by a fixed number of binary bits. Table 18-2 shows 4-bit BCD encoding.

Note that the BCD encoding is not the binary equivalent of a decimal. For example, the binary equivalent of 2018 is `0111_1110_0010`, while the corresponding BCD code is `0010_0000_0001_1000`, as shown in Figure 18-2.

Decimal Digit	BCD
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1

Table 18-2. Binary coded decimal (BCD)

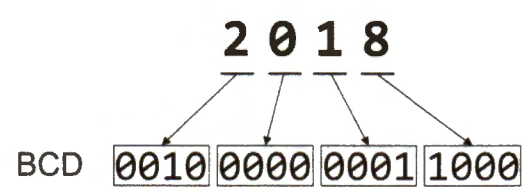


Figure 18-2. BCD encoding of 2018

In the RTC time and date registers, the unit's digit of the month, day, hour, minute, second, and last two digits of the year are encoded by two four-bit BCD codes. Some other digits are represented by only two or three bits. For example, the ten's digit for the second (ST) has a maximum value of 6, and thus only three bits are required. The date register does not record the first two digits of the year.

Figure 18-3 and Figure 18-4 shows how to set the time and date. The time can be in 12- or 24-hour format, with the PM bit indicating whether the time is in AM or PM. ST and SU stand for the second's ten's and unit's digit, respectively.

```
// Set time as 11:32:43 am
RTC->TR = 0U<<22 | 1U<<20 | 1U<<16 | 3U<<12 | 2U<<8 | 4U<<4 | 3U;
```

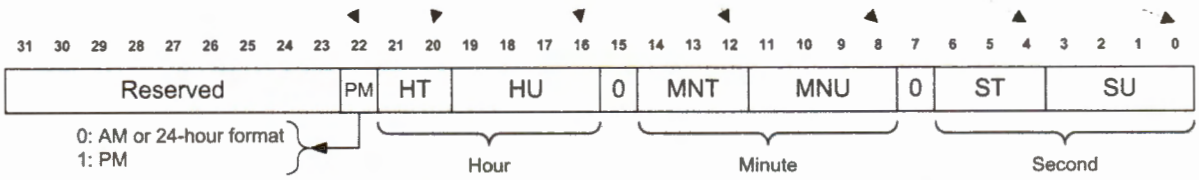


Figure 18-3. RTC time register TR (T = ten's place, U = unit's place)

```
// Set date as 2018/05/27, Sunday
RTC->DR = 1U<20 | 8U<<16 | 7U<<13 | 0U<<12 | 5U<<8 | 2U<<4 | 7U;
```

001 = Monday, ..., 111 = Sunday

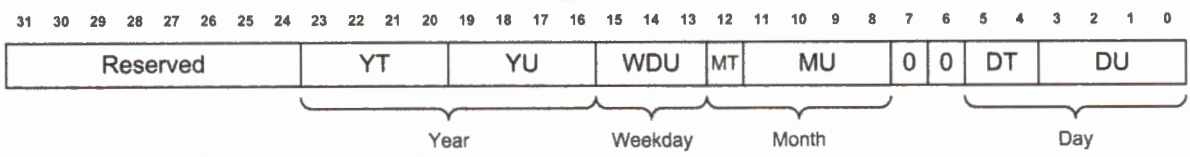


Figure 18-4. RTC date register DT (T = ten's place, U = unit's place)

Although BCD requires more bits than the binary format to represent the time and date, the BCD format is more convenient for extracting and displaying the digits of the time and date. For example, if 8-bit BCD is used, software does not have to perform division or modular operations to get the ten's digit and the unit's digit of the minute.

It is easy to decode BCD into characters for display.

18.5 RTC Initialization

By default, the RTC registers are write-protected to prevent malicious or accidental modification. Software must perform the following actions to remove write-protection.

- First, the disable backup-domain protection bit (DBP) in the power controller control register (PWR->CR1) must be set to enable write accesses to the RTC registers, as shown in Example 17-3 in Chapter 17.
- Second, software must write the keys "0xCA" and "0x53" to the RTC write protection register (RTC->WPR).

```
void RTC_Init(void) {

  // Enable RTC clock
  LCD_RTC_Clock_Enable ();  // See Example 17-3, select LSE as the clock
  RCC->BDCR |= RCC_BDCR_RTCEN; // Enable the clock of RTC

  // Disable write protection of RTC registers by writing disarm keys
  RTC->WPR = 0xCA;
  RTC->WPR = 0x53;

  // Enter initialization mode to program TR and DR registers
  RTC->ISR |= RTC_ISR_INIT;

  // Wait until INITF has been set
  while((RTC->ISR & RTC_ISR_INITF) == 0);

  // Hour format: 0 = 24 hour/day; 1 = AM/PM hour
  RTC->CR &= ~RTC_CR_FMT;

  // Generate a 1Hz clock for the RTC time counter
  // LSE = 32.768 kHz = 2^15 Hz
  RTC->PRER |= (2U<<7 - 1) << 16; // Asynch_Prescaler = 127
  RTC->PRER |= (2U<<8 - 1);       // Synch_Prescaler  = 255

  // Set time as 11:32:00 am
  RTC->TR = 0U<<22 | 1U<<20 | 1U<<16 | 3U<<12 | 2U<<8;

  // Set date as 2016/05/27
  RTC->DR = 1U<<20 | 6U<<16 | 0U<<12 | 5U<<8 | 2U<<4 | 7U;

  // Exit initialization mode
  RTC->ISR &= ~RTC_ISR_INIT;

  // Enable write protection for RTC registers
  RTC->WPR = 0xFF;
}
```

Example 18-2. Initializing RTC in C

```
        AREA RTC_Demo CODE, READONLY
        EXPORT __main
        INCLUDE stm321476xx_constants.s
        ALIGN
        ENTRY

__main
        ; Power interface clock enable
        LDR     r0, =RCC_BASE
        LDR     r1, [r0, #RCC_APB1ENR]
        ORR     r1, r1, #RCC_APB1ENR_PWREN
        STR     r1, [r0, #RCC_APB1ENR]

        ; Disable backup domain write protection
        LDR     r0, =PWR_BASE
        LDR     r1, [r0, #PWR_CR]
        ORR     r1, r1, #PWR_CR_DBP
        STR     r1, [r0, #PWR_CR]

        ; Write '0xCA' and '0x53' to unlock the write protection
        LDR     r0, =RTC_BASE
        MOV     r1, #0xCA
        STR     r1, [r0,  #RTC_WPR]
        MOV     r1, #0x53
        STR     r1, [r0,  #RTC_WPR]

        ; Select and enable LSE Clock
        LDR     r0, =RCC_BASE
        LDR     r1, [r0, #RCC_CSR]
        ORR     r1, r1, #RCC_CSR_RTCSEL_LSE
        ORR     r1, r1, #RCC_CSR_RTCEN
        ORR     r1, r1, #RCC_CSR_LSEON
        STR     r1, [r0, #RCC_CSR]

        ; Wait until LSE clock ready
Wait    LDR     r0, =RCC_BASE
        LDR     r1, [r0, #RCC_CSR]
        AND     r1, r1, #RCC_CSR_LSERDY
        CMP     r1, #0
        BEQ     Wait

        ; Generate a 1-Hz clock for the calendar counter
        LDR     r0, =RTC_BASE
        LDR     r1, [r0, #RTC_PRER]
        ORR     r1, r1, #0xFF
        ORR     r1, r1, #0x7F0000
        STR     r1, [r0, #RTC_PRER]

        LDR     r0, =RTC_BASE
        LDR     r1, [r0, #RTC_ISR]
```

```
        ORR     r1, r1, #RTC_ISR_INIT
        STR     r1, [r0, #RTC_ISR]

        ; Wait until INITF flag is set
Wait2   LDR     r0, =RTC_BASE
        LDR     r1, [r0, #RTC_ISR]
        AND     r1, r1, #RTC_ISR_INITF
        CMP     r1, #0
        BEQ     Wait2

        ; Set time as 11:32:00 am
        LDR     r0, =RTC_BASE
        MOV     r2, #0
        MOV     r1, #1                  ; 1 = pm, 0 = am
        ORR     r2, r2, r1, LSL #22
        MOV     r1, #1                  ; hour's tens
        ORR     r2, r2, r1, LSL #21
        MOV     r1, #1                  ; hour's ones
        ORR     r2, r2, r1, LSL #16
        MOV     r1, #3                  ; min's tens
        ORR     r2, r2, r1, LSL #12
        MOV     r1, #2                  ; min's ones
        ORR     r2, r2, r1, LSL #8
        STR     r1, [r0, #RTC_TR]

        ; Exit initialization mode
        LDR     r0, =RTC_BASE
        LDR     r1, [r0, #RTC_ISR]
        BIC     r1, r1, #RTC_ISR_INIT
        STR     r1, [r0, #RTC_ISR]

        ; Enable the RTC protection
        LDR     r0, =PWR_BASE
        MOV     r1, #0xFF
        STR     r1, [r0, #PWR_CR]

stop    B stop
        END
```

Example 18-3. Initializing RTC in assembly

The assembly file `stm32l476xx_constants.s` defines many constants, such as the base memory addresses:

```
RCC_BASE      EQU (AHB1PERIPH_BASE + 0x1000)
RTC_BASE      EQU (APB1PERIPH_BASE + 0x2800)
```

It also includes the byte offset from corresponding base memory addresses, such as

```
RCC_APB1ENR1 EQU 0x58
RTC_ISR      EQU 0x0C
```

18.6 RTC Alarm

The RTC module also provides alarm functions that allow the processor to execute a task at a scheduled time. For example, the RTC alarm can wake up the processor at a certain time after it has entered a low-power mode.

The STM32L RTC module has two programmable alarm units: alarm A and alarm B.

- The alarm time and date for alarm A are saved in the `RTC_ALRMAR` register, whereas for alarm B they are stored in the `RTC_ALRMBR` register. They can be written only if the `ALRAWF` flag is 1 in the `RTC_ISR` register.
- The alarm time and date are compared with the RTC date held in the `RTC_DR` register and the RTC time stored in the `RTC_TR` register.
- If enabled, the hardware module generates an RTC alarm interrupt if the time and date match.

The RTC module allows software to select the alarm time and date flexibly. The day of the week, the hour, the minute, and the second can be individually chosen by their mask bits to take part in the comparison. If a mask bit is `1`, its corresponding time component is ignored during the alarm time comparison. These masks can be used to generate periodic alarms.

Suppose the alarm time register is set to `21:25:37` on Monday. Table 18-3 shows a few examples of various settings of the mask bits.

MSK 4 (day of week)	MSK 3 (hour)	MSK 2 (minute)	MSK 1 (Second)	When does the alarm occur?
0	0	0	0	At 21:25:37 on each Monday
1	0	0	0	At 21:25:37 every day
1	1	1	0	At the 37th second of every minute
0	0	0	1	At every second of 21:25 on each Monday
1	0	0	1	At every second of 21:25 every day
0	0	1	1	At every second of the 21st hour on each Monday
0	1	1	1	At every second on Monday

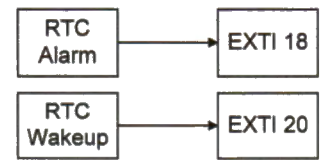
Table 18-3. Example of mask bit setting for various alarm time comparisons

The following program sets up alarm A, which occurs at the 30th second of each minute.

```
void RTC_Set_Alarm(void) {

  uint32_t AlarmTimeReg;

  // Disable alarm A
  RTC->CR &= ~RTC_CR_ALRAE;

  // Remove write-protection of RTC registers by writing "0xCA"
  // and then "0x53" into the WPR register
  RTC->WPR = 0xCA;                  // WPR: write protection register
  RTC->WPR = 0x53;

  // Disable alarm A and its interrupt
  RTC->CR &= ~RTC_CR_ALRAE;         // Clear alarm A enable bit
  RTC->CR &= ~RTC_CR_ALRAIE;        // Clear alarm A's interrupt enable bit
  // Wait until access to alarm registers is allowed
  // Write flag (ALRAWF) is set by hardware if alarm A can be changed.
  while((RTC->ISR & RTC_ISR_ALRAWF) == 0);

  // Set off alarm A if the second is 30
  // Bits[6:4] = Ten's digit for the second in BCD format
  // Bits[3:0] = Unit's digit for the second in BCD format
  AlarmTimeReg = 0x3 << 4;

  // Set alarm mask field to compare only the second
  AlarmTimeReg |= RTC_ALRMAR_MSK4;  // 1: Ignore day of week in comparison
  AlarmTimeReg |= RTC_ALRMAR_MSK3;  // 1: Ignore hour in comparison
  AlarmTimeReg |= RTC_ALRMAR_MSK2;  // 1: Ignore minute in alarm comparison
  AlarmTimeReg &= ~RTC_ALRMAR_MSK1; // 0: Alarm sets off if the second match

  // RTC alarm A register (ALRMAR)
  RTC->ALRMAR = AlarmTimeReg;

  // Enable alarm A and its interrupt
  RTC->CR |= RTC_CR_ALRAE;          // Enable alarm A
  RTC->CR |= RTC_CR_ALRAIE;         // Enable alarm A interrupt

  // Enable write protection for RTC registers
  RTC->WPR = 0xFF;
}
```

Example 18-4. Initializing alarm A to generate an alarm on the 30th second of each minute

All RTC interrupts are connected to the EXTI controller. For example, the RTC alarm signal is linked to EXTI line 18 internally, and the RTC wakeup signal is connected to EXTI line 20. More details can be found in the STM32L4 reference manual. To enable the alarm interrupt, we must enable EXTI line 18, which is connected to the RTC alarm signals.



```
void RTC_Alarm_Enable(void){

  // Other initialization (see Example 18-2)
  ...

  // Configure EXTI 18
  // Select triggering edge
  EXTI->RTSR1 |= EXTI_RTSR1_RT18; // 1 = Trigger at rising edge

  // Interrupt mask register
  EXTI->IMR1 |= EXTI_IMR1_IM18;  // 1 = Enable EXTI 18 line

  // Event mask register
  EXTI->EMR1 |= EXTI_EMR1_EM18;  // 1 = Enable EXTI 18 line

  // Interrupt pending register
  EXTI->PR1  |= EXTI_PR1_PIF18;  // Write 1 to clear pending interrupt

  // Enable RTC interrupt
  NVIC->ISER[1] |= 1<<9;          // RTC_Alarm_IRQn = 41, See Chapter 11.6.1
  // It is equivalent to: NVIC_EnableIRQ(RTC_WKUP_IRQn);

  // Set interrupt priority as the most urgent
  NVIC_SetPriority(RTC_WKUP_IRQn, 0);

  ...

}
```

Example 18-5. Connecting RTC alarm interrupt to EXTI 18

In this example, when an RTC alarm interrupt takes place, we toggle the LED connected to GPIO pin PB 2. The RTC alarm interrupt handler must clear the interrupt pending flag of EXTI line 18 and the interrupt status flag of alarm A.

```
void RTC_Alarm_IRQHandler(void){

  // RTC initialization and status register (RTC_ISR)
  // Hardware sets the Alarm A flag (ALRAF) when the time/date registers
  // (RTC_TR and RTC_DR) match the alarm A register (RTC_ALRMAR), according
  // to the mask bits

  if(RTC->ISR & RTC_ISR_ALRAF){
    GPIOB->ODR ^= 1UL<<2;          // Toggle GPIO pin PB.2
    RTC->ISR &= ~(RTC_ISR_ALRAF); // Clear the alarm A interrupt flag
  }

  // Clear the EXTI line 18
  EXTI->PR1 |= EXTI_PR1_PIF18;    // Write 1 to clear pending interrupt
}
```

Example 18-6. RTC Alarm interrupt handler

18.7 Using RTC to Wake Processors up from Sleep Mode

Power requirements are one of the most critical constraints in embedded systems, especially in mobile and portable systems. Cortex-M processors provide several operating modes, which offer various tradeoffs between energy efficiency and performance (*e.g.* processor speed and wake-up time). A mode that consumes less energy often requires a longer wake-up time.

The STM32L4 has eight power modes, as shown in Figure 18-5. After power-on or a system reset, the processor enters ***run*** mode. By changing on-chip peripherals to low-power modes or by turning off the clock to the peripherals and core, software can switch the processor to different power modes.

A typical approach is to use software to switch the processor to ***sleep*** or ***standby*** mode during idle periods, and wake the processor up when an interrupt or event occurs. In both modes, all clocks except LSI and LSE are stopped, and the processor core is turned off. Sleep and standby mode differ from each other in three major aspects.

- First, in standby mode, data in peripheral registers and SRAM are lost by default. However, data are retained in sleep mode.
- Second, peripherals are active in sleep mode but are turned off in standby mode.
- Third, when the processor exits from standby mode, a reset signal is generated internally, and the processor reboots. However, a reboot is not required when exiting from sleep mode. Therefore, the wake-up time for sleep mode is much shorter than it is for standby mode.

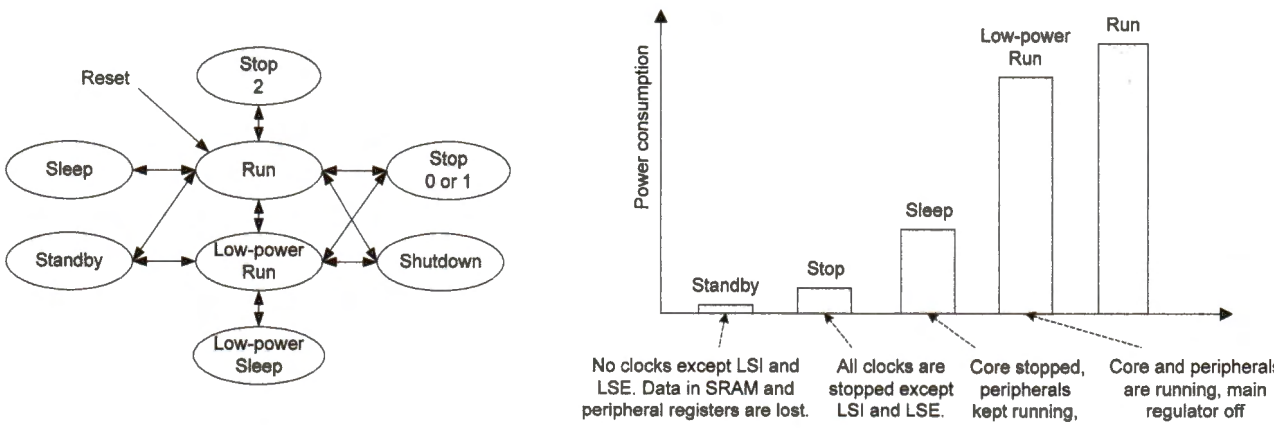


Figure 18-5. STM32L4 supports 8 power modes, which offer various performance levels and energy efficiencies.

Driven by LSI and LSE, the RTC module is an ultra-low power module which takes only a few *nA* of electric current. A small button battery can supply power to a RTC module for many years. Therefore, RTC keeps working even in standby or shutdown mode. Naturally, RTC is widely used to make the processor wake up from low-power modes periodically.

Avoid waking-up too frequently

This section demonstrates how to switch the processor to sleep mode, and how to use the RTC to wake the processor up from sleep mode. Suppose we want to wake the processor up every five seconds. Software can program the RTC wakeup time by

- selecting the clock (`WUCKSEL` bits in register `CR`), which drives the wakeup timer,
- setting the timer auto-reload value (register `WUTR`).

Example 18-7 shows a subroutine that programs the RTC wakeup timer to generate a wakeup event every 5 seconds. This subroutine can only be called after RTC write protection has been disabled (see Example 18-2).

```
void RTC_Wakeup_Configuration(void){
  // Wakeup initialization can only be performed when wakeup is disabled.
  RTC->CR &= ~RTC_CR_WUTE;     // Disable wakeup counter

  // WUTWF: Wakeup timer write flag
  // 0: Wakeup timer configuration update not allowed
  // 1: Wakeup timer configuration update allowed
  while( (RTC->ISR & RTC_ISR_WUTWF) == 0 );

  // WUCKSEL[2:0]: Wakeup clock selection
  // 10x: ck_spre (usually 1 Hz) clock is selected
  RTC->CR &= ~RTC_CR_WUCKSEL;
  RTC->CR |= RTC_CR_WUCKSEL_2;  // Select ck_spre (1Hz)

  // RTC wakeup timer register (Max = 0xFFFF)
  RTC->WUTR = 5;  // The counter decrements by 1 every pulse of
                  // the clock selected by WUCKSEL.

  // Enable wake up counter and wake up interrupt
  RTC->CR |= RTC_CR_WUTIE; // Enable wake up interrupt
  RTC->CR |= RTC_CR_WUTE;  // Enable wake up counter
}
```

Example 18-7. Programming RTC to generate a wake-up interrupt every 5 seconds

The processor enters sleep mode by executing "`__WFI()`" or "`__WFE()`", which run the assembly instruction "`WFI`" (*Wait For Interrupt*) and "`WFE`" (*Wait For Event*), respectively.

- If `__WFI()` is executed, the processor can be woken up by interrupt requests, system reset and debug operations.
- If `__WFE()` is executed, the processor can be woken up by events. Example events include interrupts, debug events, and events sent by the `SEV` (send event) instruction.

The behavior of the processor after being woken up depends on the `Sleep-on-Exit` bit in the System Control Register (`SCR`), as shown in Figure 18-6. When an interrupt request arrives, the processor is woken up from sleep mode and starts to execute the corresponding interrupt service routine.

- If the `Sleep-on-Exit` bit is 1, the function `__WFI()` does not return to its caller after the interrupt service routine completes. Instead, the processor will immediately enter sleep mode again if there are no new interrupt requests.
- If the `Sleep-on-Exit` bit is 0, the function `__WFI()` returns to its caller after the interrupt service routine completes. This allows the caller to resume the computation and execute the code after the `__WFI()` statement.

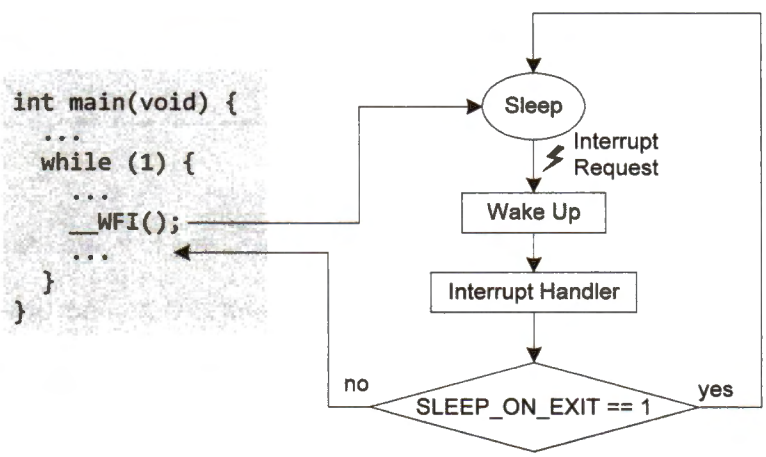


Figure 18-6. Continuous sleep if the Sleep-on-Exit bit is set

Example 18-8 shows the code that switches the processor to sleep mode. Depending on the application need, the `Sleep-on-Exit` bit can be either set or cleared. On STM32L4, the RTC wakeup interrupt is internally connected EXTI line 20. To enable RTC wakeup interrupts, we need to enable interrupts and events for EXTI line 20.

```
void Enter_SleepMode(void){

  // Cortex system control register
  SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk;    // 0 = sleep, 1 = deep sleep
```

```
  // SLEEPONEXIT: Indicates sleep-on-exit when returning from handler
  // mode to thread mode:
  //  0 = do not sleep when returning to thread mode.
  //  1 = enter sleep, or deep sleep, on return from an ISR.
  SCB->SCR &= ~SCB_SCR_SLEEPONEXIT_Msk;

  // RTC wakeup interrupt is connected to EXTI line 20 internally.
  // Enable EXTI20 interrupt
  EXTI->IMR1 |= EXTI_IMR1_IM20;

  // Enable EXTI20 event
  EXTI->EMR1 |= EXTI_EMR1_EM20;

  // Select rising-edge trigger
  EXTI->RTSR1 |= EXTI_RTSR1_RT20;

  NVIC_EnableIRQ(RTC_WKUP_IRQn);
  NVIC_SetPriority(RTC_WKUP_IRQn, 0);

  __DSB();  // Ensure that the last store takes effect
  __WFI();  // Switch processor into the sleep mode
}
```

Example 18-8. Entering the sleep mode by using `WFI`

18.8 Exercises

1. Write a C program that converts the UNIX Epoch time to a calendar date and time.
2. Write a C program that converts a given calendar date and time to the UNIX Epoch time.
3. Write an assembly program that sets up the RTC date and displays the current date on an LCD.
4. Write an assembly program that sets up the RTC time and displays the current time on an LCD.
5. Implement an assembly program to allow users to change the date and time via the keypad.
6. Write an assembly program that utilizes the RTC alarm to toggle an LED every five seconds.